Benchmarking Multimodal LLM Agents for GUI Automation across Cloud and Local Settings

Anh-Nhat Nguyen[2034311]

University of Mannheim, Germany

Abstract. *We present a benchmarking study of multimodal agents for GUI automation, comparing local and cloud-based configurations on a diverse set of desktop interaction tasks. Our initial pipeline combined OmniParser with a local GPT-4o-mini model but struggled with resource constraints, instability, and limited task success. To address these limitations, we developed a cloud-based system using Anthropic Claude 3.5, supported by automated evaluation, retry management, and a Streamlit-based review tool. The agent was tested on 20 tasks spanning five functional domains, revealing key failure modes such as scroll misalignment, element misidentification, and fragile form flows. Prompt-level refinements, like Tab-key navigation and explicit step plans—converted many partial failures into robust successes. The final setup achieved 56% task success, 89% step accuracy, and 57% action efficiency. Our findings outline trade-offs between local and cloud agents and provide practical guidance for designing reliable, privacy-aware GUI automation systems.*

Keywords: Multimodal, LLM Agents, GUI Automation, Cross-platform, Open-source, Prompt Engineering, Autonomous Agents.

1 Introduction

1.1 Problem Statement

Modern GUI automation agents promise to liberate users from repetitive workflows by interpreting natural language commands and interacting directly with applications. However, most state-of-the-art systems rely on proprietary, cloud-hosted vision-language APIs such as GPT-4V or Gemini Vision to parse screens and generate actions. This dependence raises concerns around privacy, cost, and vendor lock-in.

Meanwhile, fully local solutions that pair self-hosted LLMs with open-source UI parsers often struggle under real-world conditions. They suffer from sluggish response times, memory limitations, and brittle handling of dynamic or multi-step tasks.

In this work, we benchmark the performance of multimodal GUI agents across both local and cloud-based configurations. We quantify task-level and step-level success on a representative suite of 20 real-world desktop and web tasks, isolate principal failure modes through systematic error analysis, and explore prompt refinement techniques that can convert brittle behaviors into robust executions. Our findings highlight the trade-offs between local deployment and

scalable cloud-based solutions, offering practical insights into cost, robustness, and reliability in GUI automation.

1.2 Related Work

Computer control agents (CCAs) represent a growing class of artificial intelligence systems designed to operate computing environments, such as desktops, mobile platforms, or web interfaces through natural language instructions. These agents must perceive dynamic user interfaces, interpret high-level goals, and perform sequences of low-level actions (e.g., clicks, keystrokes) to complete tasks. This problem domain lies at the intersection of GUI automation, instruction following, and generalist agent design. In this section, we review the evolution of CCA research, focusing on methodological developments, core system architectures, and evaluation frameworks.

Early Approaches: Initial research into instruction-based agents primarily adopted reinforcement learning (RL) paradigms in controlled environments. For instance, [1] used RL to map documentation to editor commands, showing early signs of grounding language in action. This line of work evolved into environments like MiniWoB [2], which offered standardized, low-complexity web tasks for instruction following. However, these early agents were typically specialized for specific domains, dependent on handcrafted observation and action spaces. They performed reasonably well in narrow settings but lacked the scalability and temporal reasoning required for complex, real-world GUI tasks.

The Shift to Foundation Models: The introduction of large-scale foundation models, particularly large language models (LLMs) and vision language models (VLMs), has shifted the paradigm from task-specific to more generalist agents. These models offer powerful out-of-the-box capabilities for language understanding and reasoning. Rather than training policies from scratch, recent agents use pre-trained models to interpret tasks and plan action sequences. This transition is often analyzed in three dimensions: the agent's environment, its mode of interaction, and its internal architecture [3, 4]. For example, agents may observe the environment visually (screenshots), structurally (HTML), or via accessibility APIs, and may act through pixel-level controls or direct API calls.

Concrete examples include WebGPT [5], which augments an LLM with a Browse interface, and Mind2Web [6], which generalizes task execution across unseen websites. While these models showcase impressive generalization, their reliance on large, proprietary APIs often entails significant computational cost and raises data privacy concerns, motivating the exploration of more accessible and self-contained alternatives.

Architectural Components and Memory: As CCAs grow in capability, modular architectures have become increasingly common. A typical agent pipeline

includes components for observation encoding, instruction parsing, action planning, and execution. Many systems integrate these components with a central LLM using techniques like chain-of-thought prompting [7]. The literature distinguishes between memoryless agents, history-based agents, and state-based agents depending on how they encode temporal context [8–10]. Many current agents overlook long-term memory, relying only on recent observations. This limitation, often coupled with a dependency on a cloud-hosted model for reasoning, contributes to brittle performance and reinforces the cost and privacy challenges we aim to address.

Evaluation and Benchmarks: The field has also matured in its evaluation methodologies. Early benchmarks like MiniWoB++ [2] were suitable for RL but inadequate for testing generalization. More recent benchmarks, such as Mind2Web and WebArena [11], offer open-domain tasks on realistic websites, enabling the assessment of zero-shot performance and robustness. The community has developed a range of metrics beyond simple task success, including step-level accuracy, action grounding error, and robustness to UI changes [12–14]. Standardized and reproducible evaluation setups are crucial for fairly comparing different approaches, especially when assessing trade-offs between proprietary models and open-source solutions.In this work, we adopt three evaluation metrics to assess overall task success, step-level accuracy, and action efficiency. For detailed definitions and formulas, see Section 2.4.

Recent Advances and Lingering Gaps Recent developments continue to push the boundaries of what CCAs can achieve. Anthropic, for instance, demonstrated that its Claude 3.5 model can operate computers in a sandboxed environment using only visual input and simulated controls, mirroring human-like interaction [15]. While technically impressive, this capability remains locked within a proprietary, closed-source model, highlighting the exact vendor lock-in problem this paper seeks to solve.

In parallel, progress in interface abstraction, such as Microsoft's OmniParser [16], offers a more open path forward. OmniParser creates a unified, layout-aware encoding for GUIs that is model-agnostic. This type of modular, interface-level standardization is a key enabler for building robust agents that are not tied to a specific foundation model, aligning directly with our goal of promoting platform versatility.

In summary, research in computer control has evolved from specialized, RL-based agents to generalist systems powered by foundation models. While these modern agents demonstrate remarkable capabilities, they often inherit limitations from the proprietary models they depend on, such as high operational costs and privacy risks. Key challenges remain in long-horizon planning, robust action grounding, and adapting to novel interfaces. The trend toward modular components like OmniParser suggests a viable path toward building more open and flexible systems. This work builds on that trajectory by developing and benchmarking a multimodal agent architecture centered around the Claude

Computer Use pipeline from Anthropic, focusing on execution reliability, prompt refinement strategies, and detailed error analysis across diverse GUI automation tasks.

1.3 Structure

This report is organized as follows. Section 1 introduces the problem and related work. Section 2 outlines our experimental setup, task dataset, agent pipelines, and evaluation metrics. Results and error analysis are presented in Section 3, followed by a discussion of findings and future directions in Section 4, coding source included in Section 5.

2 Experimental Design

2.1 Dataset

To evaluate the agent's capabilities across real-world computing scenarios, we curated a benchmark suite of 20 desktop interaction tasks. Instead of adopting full-scale standard benchmarks such as WebArena or Mind2Web, we selected a compact subset for practical and methodological reasons. First, commercial API costs limited the feasibility of large-scale evaluation, especially for vision-language agents like Claude 3.5. Second, our analysis emphasizes trajectory-level breakdowns, including retries and failure diagnostics, which are more tractable with a smaller, well-controlled dataset.

To ensure relevance and comparability, we followed the category structure used in existing benchmarks, covering five functional domains: (1) Basic App/-File Operations, (2) Programming and Scripting, (3) Communication and Forms, (4) Web Automation, and (5) System Monitoring and Data Extraction. Each domain reflects a distinct user interaction modality commonly encountered in desktop environments. Task difficulty ranges from Easy to Hard, and each task includes a natural language instruction, a domain label (numerically encoded), and a zero-shot prompt suitable for LLM-based agents. The full task list is provided in Table 1.

We also manually annotated the Optimal Actions for each task, enabling precise computation of Action Efficiency (AE) (see Section 2.4). For example, Task 1 (*Launch Calculator and compute* $123 + 456$) requires four optimal steps: open Calculator, enter "123+456," press "=," and read the result.

2.2 Local LLM Agent

In the initial phase of our study, we developed a purely local agent by coupling OmniParser [1] with a locally hosted instance of GPT-4o-mini. OmniParser operated on high-resolution macOS screenshots, performing object detection to identify UI elements (including bounding boxes, element types, and any visible

[1] OmniParser: Screen Parsing tool for Pure Vision Based GUI Agent

text) and emitting a structured JSON representation of the current interface. This JSON payload, together with the user's natural-language instruction, was fed into GPT-4o-mini, which generated a sequence of executable actions (e.g., `click(x,y)`, `type("...")`) to accomplish the task, the architecture shown in 1. The virtual environment is based on macOS Sequoia, deployed using the virtual machine provided by CUA[2].

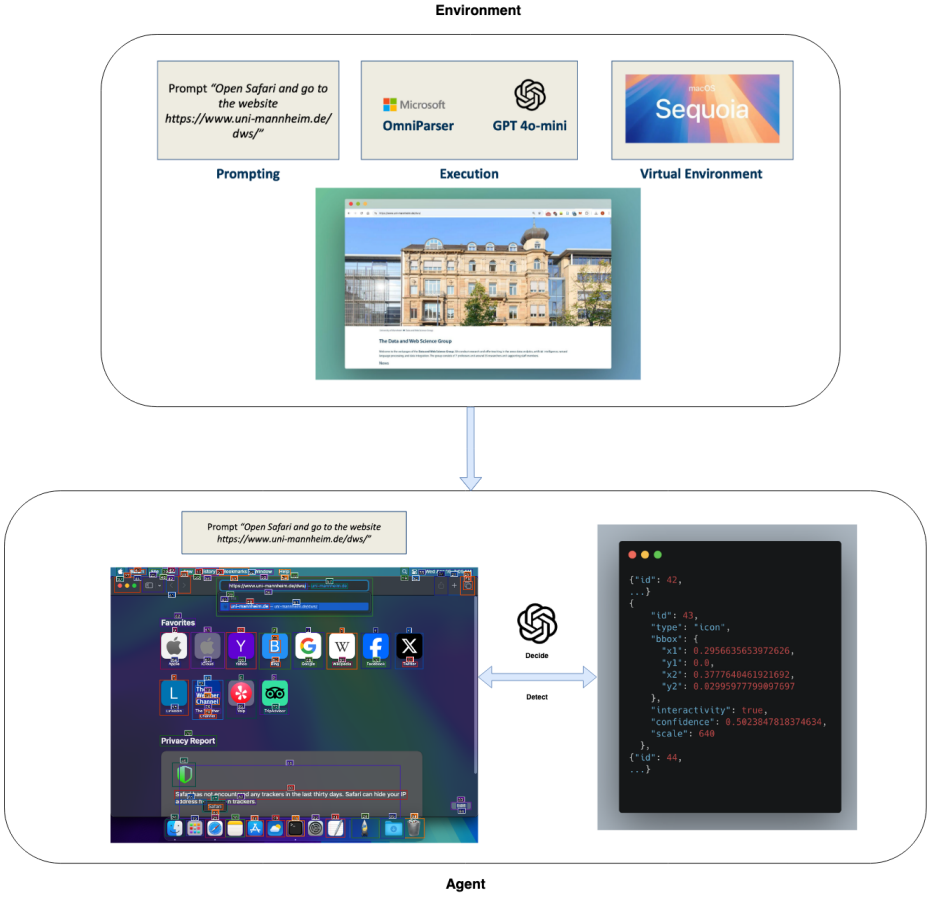


Fig. 1: Architecture for experimental design.

However, during experimentation we encountered severe performance bottlenecks. Inference with GPT-4o-mini on a local machine Apple Macbook M1 Pro consumed nearly 100% of available CPU resources and exhausted 16 GB of RAM, resulting in repeated out-of-memory failures and multi-minute runtimes

[2] Docker for Computer-Use Agents

per task. These technical limitations prevented reliable computation of finer-grained metrics such as Action Efficiency (AE) and Step Success Rate (SSR), since each evaluation run frequently timed out or crashed before completion.

Accordingly, we restricted our evaluation to the Task Success Rate (TSR) in 2.2, the percentage of instructions fully executed to termination. We benchmarked the Local LLM Agent on dataset 1. The agent achieved an overall TSR of 40%, with performance stratified as 60% on easy tasks, 30% on medium tasks, and 20% on hard tasks,

To further gauge robustness, we then evaluated the same local agent on an additional set of production-like tasks drawn from the Claude Computer Use benchmark. In this extended evaluation, the Local LLM Agent failed to complete any of the new tasks, yielding a 0% TSR. Given these underwhelming results, compounded by prohibitive computational demands and unstable execution, we discontinued the local-only approach and migrated our evaluations to the cloud-based Claude Computer Use system, which provides higher model capacity, more efficient scaling, and greater stability for complex GUI automation tasks

Task Success Rate (TSR) Out of the 20 tasks, the local pipeline fully completed $N_{\text{succ}} = 7$ before a terminal failure, yielding

$$\text{TSR}_{\text{local}} = \frac{7}{20} \times 100 = 40\%.$$

2.3 Anthropic Claude for Computer Use

Building on the local LLM environment described in Section 2.2, we constructed a hybrid evaluation pipeline around Anthropic Claude 3.5 [3] and a Streamlit-based interface. Key features include:

- **Automatic Evaluation:** Each execution trace is fed back into Claude 3.5, which using a standardized prompt labels each step as correct or incorrect and predicts overall task success. This automated scoring accelerates throughput and provides a consistency check against human annotations.
- **Streamlit Review System:** A custom Streamlit application (run_eval.py) presents screenshots and command logs in sequence, allows evaluators to confirm or override Claude's labels, and captures optimal action counts. Interactive charts of TSR, SSR, and AE update in real time.
- **Retry Management:** On task failure, the system can automatically requeue the same instruction with modified prompts or environmental tweaks (e.g. explicit focus hints), up to a configurable retry limit, enabling systematic prompt refinement experiments.
- **Trajectory Logging:** All agent–environment interactions—including natural-language instructions, Claude's responses, GUI/API actions, and full-screen

[3] Claude for Computer Docs

snapshots—are recorded in a structured JSON trajectory. These logs support reproducible replay, error analysis, and offline batch evaluation via a CLI tool (eval_cli.py).

This architecture unifies high-volume automated scoring with human-in-the-loop oversight, supports iterative prompt engineering through managed retries, and ensures that every step of the agent's behavior is captured for detailed post-hoc analysis. The example for an interaction between human, LLM and weather API show in figure 2:



Fig. 2: Example of test case 14 - interact with weather API.

2.4 Evaluation Metrics

To assess agent performance under realistic and constrained computing conditions, we adopt three principal evaluation metrics: **Task Success Rate (TSR)**, **Step Success Rate (SSR)**, and **Action Efficiency (AE)**. These metrics are widely used in the literature [6, 17–19] and provide complementary perspectives on the correctness, completeness, and efficiency of agent behavior.

Task Success Rate (TSR) measures how often the agent completes an entire task without a critical error. In settings where each of our T tasks is executed R times (e.g. different seeds or starting conditions), we treat each run as an independent trial. This lets us capture variability: a task that sometimes succeeds and sometimes fails contributes proportionally.

$$\mathrm{TSR} = \frac{1}{T\,R} \sum_{j=1}^{T} \sum_{r=1}^{R} \mathbf{1}\{\text{task } j \text{ succeeded on run } r\} \;\times\; 100.$$

Here $\mathbf{1}\{\cdot\}$ is the indicator function (1 if true, 0 otherwise). Reporting TSR in "mean ± std dev" or with confidence intervals then directly reflects run-to-run variability.

Step Success Rate (SSR) (also called action accuracy or partial match) evaluates how many of the agent's individual actions align with a reference trajectory. By summing over all tasks and runs, we avoid biasing toward tasks with more or fewer steps:

$$\mathrm{SSR} = \frac{\sum_{j=1}^{T} \sum_{r=1}^{R} N_{\text{correct steps}}^{(j,r)}}{\sum_{j=1}^{T} \sum_{r=1}^{R} N_{\text{total steps}}^{(j,r)}} \;\times\; 100.$$

A high SSR (e.g. > 90 %) indicates reliable, per-action accuracy even if entire tasks sometimes fail.

Action Efficiency (AE) measures how concisely the agent reaches success, penalizing redundant or mistaken steps. Again we aggregate over tasks and runs:

$$\mathrm{AE} = \frac{\sum_{j=1}^{T} \sum_{r=1}^{R} N_{\text{useful actions}}^{(j,r)}}{\sum_{j=1}^{T} \sum_{r=1}^{R} N_{\text{total actions}}^{(j,r)}} \;\times\; 100.$$

3 Results and Analysis

3.1 Overall Evaluation

Across 20 distinct tasks, we performed 34 total executions—including manual retries to refine prompts or adjust the environment. Of these 34 runs, 19 succeeded and 15 failed, yielding an aggregate Task Success Rate of 0.56. At the

task level, 18 tasks succeeded at least once, while 2 tasks (IDs 5 and 11) failed in every attempt.

The agent's performance on these 34 executions is summarized in Table 2. Although individual action precision is strong, end-to-end success rate and efficiency metrics reveal significant gaps that merit further investigation.

Table 2: Overall Evaluation Metrics (34 executions across 20 tasks)

Metric	**Mean**	Std. Dev.	Min	25%	Median	75%	Max
TSR	**0.56**	0.50	0.00	0.00	1.00	1.00	1.00
SSR	**0.89**	0.24	0.00	0.96	1.00	1.00	1.00
AE	**0.57**	0.30	0.00	0.33	0.50	0.73	1.00
Total Steps	9.59	7.74	0.00	4.00	6.00	14.75	27.00
Actual Actions	7.26	5.72	0.00	4.00	4.00	12.75	20.00
Optimal Actions	4.21	1.49	1.00	3.25	4.00	6.00	6.00

The Task Success Rate is only moderate (TSR = $\mathbf{0.56} \pm 0.50$), meaning that 56% of execution attempts finish successfully while 44% fail outright. Because each run's TSR is strictly 0 or 1, even a few incorrect or misordered actions can cause the entire task to fail.

The mean Step Success Rate (SSR = $\mathbf{0.89} \pm 0.24$) demonstrates that the agent correctly executes individual low-level operations in nearly 9 out of 10 cases. Such a high SSR is on par with or exceeds figures reported in prior GUI automation research, where per-action accuracies typically range from 0.75 to 0.85.

Action Efficiency further underscores inefficiencies: AE = $\mathbf{0.57} \pm 0.30$ means the agent takes nearly twice the optimal number of actions on average (mean Actual = 7.26 vs. Optimal = 4.21). The wide interquartile range (IQR = 0.33–0.73) reveals that while some executions are near-optimal, others suffer severe redundancy, suggesting that the agent's planning heuristics lack consistency and may over-explore GUI states or fail to exploit affordances effectively.

3.2 Evaluation by Category

Table 3 breaks down performance across five domain categories (n=34 tasks, categories available for 34).

Table 3: Performance Metrics by Task Category

Category	n	TSR (mean $\pm$ SD)	SSR (mean $\pm$ SD)	AE (mean $\pm$ SD)
Basic App/File Operations	9	0.78 ± 0.44	0.85 ± 0.24	0.65 ± 0.34
Communication & Forms	10	0.20 ± 0.42	0.90 ± 0.32	0.56 ± 0.21
Programming/Script Tasks	3	1.00 ± 0.00	0.69 ± 0.34	0.75 ± 0.43
System Monitoring / Data Extraction	3	1.00 ± 0.00	0.92 ± 0.14	0.61 ± 0.35
Web Automation	9	0.44 ± 0.53	0.96 ± 0.11	0.42 ± 0.27

Category-Level Findings

- **Basic App/File Operations** (n = 9) achieve relatively high end-to-end success (TSR = 0.78 ± 0.44) but still display moderate step errors (SSR = 0.85 ± 0.24) and inefficiency (AE = 0.65 ± 0.34). Improving UI-state perception and path-planning could reduce redundant steps.
- **Communication & Forms** (n = 10) suffer the lowest TSR (TSR = 0.20 ± 0.42) despite strong per-step accuracy (SSR = 0.90 ± 0.32). This decoupling underscores the need for robust sequence-control and field-validation mechanisms in form-filling and text-entry contexts.
- **Programming/Script Tasks** (n = 3) reach perfect end-to-end success (TSR = 1.00 ± 0.00) but show the lowest step success (SSR = 0.69 ± 0.34), indicating challenges with precise syntactic operations despite coarse success criteria. Grammar-aware action models may improve reliability.
- **System Monitoring / Data Extraction** (n = 3) also hit perfect success (TSR = 1.00 ± 0.00), with high SSR (SSR = 0.92 ± 0.14) and moderate AE (AE = 0.61 ± 0.35). Deterministic environments facilitate robust performance, suggesting that improved environment modeling directly boosts accuracy and efficiency.
- **Web Automation** (n = 9) yields mid-range TSR (TSR = 0.44 ± 0.53) but the highest SSR (SSR = 0.96 ± 0.11) and lowest AE (AE = 0.42 ± 0.27). Dynamic page layouts and asynchronous content allow correct individual actions but compound small errors into brittle workflows—advanced DOM-aware planning and checkpointing are needed.

3.3 Error Analysis

We performed a targeted error analysis on tasks where the agent failed, aiming to identify recurring patterns and root causes. Table 4 summarizes the five principal failure cases, each labeled with a descriptive hint. The subsequent discussion elaborates on how these failures occurred and what design or runtime changes could mitigate them.

Table 4: Key Failure Modes by Task

ID – Task	Error Mode	Underlying Cause
5 – Using Safari to extract data on Web	Scroll overshoot	No element visibility check after scrolling
8 – Inserting receipt title in an email	Element misidentification	Fragile selector and missing focus verification
11 – Solving CAPTCHA on 2captcha demo site	Secure widget limitation	No integration with CAPTCHA solver or human fallback
13 – Filling online payment form	Submission blocked	No balance check before form submission
19 – Checking system CPU/RAM status	Instruction ambiguity	Unclear specification of preferred execution environment

Task 5 – Missed Content Due to Fixed Scrolling: The agent scrolled in fixed increments, which caused it to overshoot and miss the target section. This issue could be mitigated by adopting adaptive scrolling strategies that check element visibility after each scroll step.

Task 8 – Element Identification and Focus Navigation: The agent appended text to the wrong field due to imprecise selectors. The issue was resolved by using a Tab-based focus strategy. Combining strong selectors (e.g., XPath or ARIA) with explicit focus checks would improve input accuracy.

Task 11 – CAPTCHA and Secure Widget Limitations: The agent failed to solve the CAPTCHA despite several prompt variations. Since these widgets are intentionally resistant to automation, successful handling likely requires dedicated solver APIs or human-in-the-loop fallback mechanisms.

Task 13 – Form Completion Blocked by Credit Limit: The agent encountered payment blocks and failed to submit the form due to low credit. This was not a planning issue, but a missing runtime verification. Real-time balance checks or pre-submission validation would prevent such failures.

Task 19 – Misaligned Execution Modality: Instead of executing shell commands as instructed, the agent defaulted to Python scripting. This divergence stems from ambiguous prompt phrasing. Clarifying both the expected language and environment in the instruction can guide the agent to the correct toolchain.

These error modes, including scroll misalignment, weak element selection, secure widget constraints, lack of credit validation, and prompt ambiguity—highlight critical areas for improving adaptive control, runtime verification, and prompt precision in multimodal GUI automation systems.

3.4 Case Study Analysis

Iterative Refinement and Partial Successes To explore how prompt engineering and minimal code adjustments can resolve agent failures, we examined three tasks that initially failed but ultimately succeeded after refinement.

Task 8 – Email Composition in Mail App: The initial prompt (*"Compose and send an email"*) led the agent to input text into the recipient field instead of the message body. A second attempt using *"fill the content in the correct textbox"* improved targeting but still misplaced the subject line. Only after supplying an explicit, step-by-step plan (*"Open Mail → New Message → Click 'To' → Enter recipient → Click 'Subject' → Enter subject → Click body → Enter body → Send"*) did the agent complete the task correctly. This illustrates the effectiveness of detailed, sequenced instructions in resolving field-targeting errors.

Task 13 – Microsoft Forms Filling: The agent initially stalled with an "invalid URL" error. Adding *"slowly scroll down"* revealed hidden fields, enabling partial progress, but a required checkbox was still missed. Refining the prompt with a Tab-key navigation strategy (*"use Tab to traverse each input field in order"*) resulted in a successful submission. This case highlights how focus-agnostic traversal can increase reliability in dynamic or scroll-heavy web forms.

Task 19 – System Resource Plotting: A Python script failed when attempting to write output directly to ' /Desktop', triggering a 'FileNotFoundError'. Modifying the script to save the plot locally before moving it to the Desktop resolved the issue. This example demonstrates that small clarifications in environment setup can resolve otherwise brittle failures, especially in file system operations.

Persistent Failures Two tasks remained unsolvable despite extensive prompt variations, revealing fundamental limitations that prompt engineering alone could not address.

Task 5 – Safari News Extraction: The agent failed across multiple prompt variants, including basic extraction, conditional scrolling, and *"slowly scroll down until you see the News section"*. In all cases, the agent missed the relevant dynamic content. The lack of viewport feedback or element-visibility checking prevented reliable localization, suggesting a need for visual context awareness or direct DOM access.

Task 11 – CAPTCHA Solving on 2captcha Demo: Six prompt strategies—including role reframing (*"You are a white-hat hacker"*), click instructions (*"Click to Verify"*), and switching to reCAPTCHA v2—were all rejected due to policy enforcement. The agent's refusal behavior shows that secure widgets cannot be bypassed through prompts alone. Solutions likely require external solver APIs or human-in-the-loop fallback mechanisms.

These case studies support two conclusions. First, detailed plans and environment-specific clarifications can convert partial failures into consistent successes. Second, certain capability gaps, such as handling dynamic content and overcoming security constraints—require architectural changes or external integration rather than additional prompt tuning.

3.5 API Cost

This study assumes the use of **Claude 3.5 Sonnet**, a multimodal model by Anthropic. As of mid-2025, pricing is approximately $3.00 per million input tokens, $15.00 per million output tokens, and $0.01 per image. A typical task uses 765 image tokens, 600 input tokens, and 1,000 output tokens, yielding an estimated cost of **$0.28** per task.

Across 34 tasks, the total cost was **$9.52**. *Communication Forms* was the most expensive category, due to multi-field inputs and frequent failures. Medium and high complexity tasks, often with low action efficiency, accounted for most of the cost.

These results highlight the need for cost-aware benchmarking. While commercial APIs offer strong performance, their scaling costs require careful budgeting and efficient, domain-adapted agent design.

4 Discussion and Conclusion

4.1 Summary of Findings

This study benchmarked two GUI automation pipelines: a local OmniParser + GPT-4o-mini agent and a cloud-based Claude 3.5 system. The local setup was constrained by resource limitations and instability, achieving only 40% Task Success Rate (TSR). In contrast, the Claude-based pipeline—with Streamlit orchestration, automated scoring, and retry management, achieved 56% TSR, 89% Step Success Rate (SSR), and 57% Action Efficiency (AE) across 20 tasks.

Several practical insights emerged. Tab-key navigation significantly improved form-filling reliability. Explicit, step-by-step prompts helped resolve targeting errors and stabilize execution. Claude also showed strong domain-specific behavior: it consistently preferred Bash commands and performed exceptionally well in programming and system monitoring tasks, achieving perfect TSR in those categories. These results highlight Claude's strength in deterministic environments where goals and toolchains are well-defined.

Nonetheless, key challenges remain. Cloud APIs introduce recurring cost and vendor lock-in. CAPTCHAs are unsolvable via prompting alone and require external solver integration or human fallback. The lack of DOM or accessibility hooks limits fine-grained element interaction, and long-horizon tasks still suffer from error accumulation.

While the cloud-based pipeline demonstrated strong performance, especially in deterministic tasks like programming and system monitoring, it also revealed critical limitations. Tasks involving secure widgets, dynamic layouts, or multi-step planning remain fragile. These failures point to the need for richer interaction signals, such as DOM-level feedback or accessibility metadata, as well as planning mechanisms that can manage longer task horizons. Given the high cost and privacy concerns of proprietary APIs, hybrid strategies that offload simpler perception or parsing tasks locally—while reserving cloud inference for complex reasoning—may offer a more balanced approach. Expanding the benchmark to include mobile and legacy environments will help test generalization and adaptability across broader use cases.

4.2 Limitations and Future Work

While the Claude-based pipeline performed well overall, several limitations were evident. Reliance on proprietary APIs raises concerns about cost, reproducibility, and vendor lock-in. The agent was unable to solve secure UI components like CAPTCHAs, highlighting the need for external integration or human fallback. Additionally, the lack of interface-level feedback—such as DOM structure or element visibility limited robustness in dynamic tasks. Long-horizon scenarios also exposed the limitations of flat prompting, where small errors compound over multiple steps.

Future work should focus on addressing these bottlenecks through richer environment access (e.g., DOM or accessibility APIs), planning mechanisms that support multi-step memory, and selective hybrid architectures that combine local parsing with cloud reasoning. Expanding the benchmark to include mobile and legacy desktop GUIs will help assess the generalizability of these systems across diverse platforms.

5 Technical Detail

This study was conducted as part of the CS 715 seminar on solving complex tasks using large language models at the University of Mannheim[4]. A complete implementation, including source code and technical setup, is available in the accompanying GitHub repository[5].

[4] CS 715 seminar course page
[5] CS 715 ComputerUse GitHub repo

References

1. S. Branavan, H. Chen, L. Zettlemoyer, and R. Barzilay, "Reinforcement learning for mapping instructions to actions," in *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP* (K.-Y. Su, J. Su, J. Wiebe, and H. Li, eds.), (Suntec, Singapore), pp. 82–90, Association for Computational Linguistics, Aug. 2009.
2. T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, "World of Bits: An Open-Domain platform for web-based agents," in *International Conference on Machine Learning*, pp. 3135–3144, 2017.
3. L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
4. P. Sager, B. Meyer, P. Yan, R. von Wartburg-Kottler, L. Etaiwi, A. Enayati, G. Nobel, A. Abdulkadir, B. F. Grewe, and T. Stadelmann, "AI agents for computer use: A review of instruction-based computer control, GUI automation, and operator assistants," *CoRR*, vol. abs/2501.16150, 2025.
5. R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "Webgpt: Browser-assisted question-answering with human feedback," *CoRR*, vol. abs/2112.09332, 2021.
6. X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
7. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
8. R. Sutton and A. Barto, *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series, MIT Press, 2018.
9. S. J. Russell and P. Norvig, *Artificial intelligence: a modern approach*. pearson, 2016.
10. B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "Gpt-4v(ision) is a generalist web agent, if grounded," in *Proceedings of the 41st International Conference on Machine Learning*, ICML'24, JMLR.org, 2024.
11. S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *The Twelfth International Conference on Learning Representations*, 2024.
12. Z. Wu, Z. Wu, F. Xu, Y. Wang, Q. Sun, C. Jia, K. Cheng, Z. Ding, L. Chen, P. P. Liang, and Y. Qiao, "OS-ATLAS: Foundation action model for generalist GUI agents," in *The Thirteenth International Conference on Learning Representations*, 2025.
13. F. Huq, Z. Z. Wang, F. F. Xu, T. Ou, S. Zhou, J. P. Bigham, and G. Neubig, "CowPilot: A framework for autonomous and human-agent collaborative web navigation," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)* (N. Dziri, S. X. Ren, and S. Diao, eds.), (Albuquerque, New Mexico), pp. 163–172, Association for Computational Linguistics, Apr. 2025.

14. D. Nathani, L. Madaan, N. Roberts, N. Bashlykov, A. Menon, V. Moens, A. Budhiraja, D. Magka, V. Vorotilov, G. Chaurasia, D. Hupkes, R. S. Cabral, T. Shavrina, J. Foerster, Y. Bachrach, W. Y. Wang, and R. Raileanu, "MLGym: A New Framework and Benchmark for Advancing AI Research Agents," Feb. 2025. arXiv:2502.14499 [cs].
15. Anthropic, "Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku," June 2024. Accessed: 2025-07-04.
16. W. Yu, Z. Yang, J. Wan, S. Song, J. Tang, W. Cheng, Y. Liu, and X. Bai, "OmniParser V2: Structured-Points-of-Thought for Unified Visual Text Parsing and Its Generality to Multimodal Large Language Models," Feb. 2025. arXiv:2502.16161 [cs].
17. J. Zhang, J. Wu, Y. Teng, M. Liao, N. Xu, X. Xiao, Z. Wei, and D. Tang, "Android in the zoo: Chain-of-action-thought for gui agents," *arXiv preprint arXiv:2403.02713*, 2024.
18. H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, "Autodroid: Llm-powered task automation in android," in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pp. 543–557, 2024.
19. J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems* (A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, eds.), vol. 37, pp. 50528–50652, Curran Associates, Inc., 2024.

Table 1: Computer Use Task Dataset

ID	Difficulty	Task	Category	Prompt
1	Easy	Launch the Calculator app (via Spotlight or Launchpad) and compute 123 + 456.	1	Launch Calculator, perform 123 + 456, then report the result.
2	Easy	In Finder, go to your Documents folder and create a new plain-text file named experiment_notes.txt.	1	Create ~/Documents/experiment_notes.txt.
3	Easy	Open System Settings → bluetooth and turn it on	1	Turn on bluetooth
4	Easy	Use the macOS screenshot utility to capture the active window	1	Save to ~/Desktop/screenshot.png
5	Easy	Navigate to university website and verify page title contains "Data Science."	1	Visit `https://www.uni-mannheim.de/dws/` and extract title
6	Medium	In Terminal, rename all files by prepending today's date	2	Shell script to rename using YYYY-MM-DD_ format
7	Medium	Write a Python script that computes 123 + 456	2	Create plus.py and write Python program to compute sum
8	Medium	Send an email with subject and body	3	Compose email: Subject = "CS715_Test", Body = "This is a demo."
9	Medium	Log in at saucedemo.com and verify Logout is visible	4	Navigate, login, and confirm logout button visible
10	Medium	Copy data.csv and confirm size match	1	Copy file and compare size using ls -l
11	Medium	Solve CAPTCHA on demo site	3	Go to `2captcha.com/demo/geetest-v4` and solve
12	Medium	Schedule a script to extract news from university site	2	Use cron to run script and parse HTML for <news> content
13	Medium	Fill out and submit a Google Form	4	Load form, fill fields, and submit
14	Medium	Fetch weather using API and save JSON	4	Fetch from `goweather.xyz` and save result on Desktop
15	Medium	Create desktop shortcut (alias) for an app	1	Create alias for [App] on Desktop
16	Hard	Delete a file via Terminal and confirm deletion	1	Use 'rm' and confirm with 'ls'
17	Hard	Extract text from a PDF and export to CSV	5	Use script to extract and convert PDF to CSV
18	Hard	Ping Google 5 times and compute avg RTT	5	Run 'ping', parse RTTs, and average excluding failures
19	Hard	Log CPU/memory every 5s and plot results	5	Script to monitor, plot with matplotlib, and save image
20	Hard	Search "Headphones" on Amazon and report top 3 items	4	Extract name, price, reviews, shipping from top results

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt habe.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
ChatGPT	Rephrasing	Throughout	+
DeepL	Translation	Throughout	-
ResearchGPT	Summarization of related work	Sec. 2	+
Claude 4	Code generation	Sec. 2	++
ChatGPT	Related work hallucination	Most of bibliography	++
Google Gemini	Deep Research for related work		++
Github Copilot	Code generation		++

Unterschrift
Mannheim, 04.07.2025